

Documentación del Proyecto Housing

1. Estructura del Proyecto y Arquitectura

1.1 Estructura de Carpetas

El proyecto sigue una **Arquitectura Hexagonal (Ports and Adapters)**, también conocida como Clean Architecture. Esta estructura se organiza en 3 capas principales:

```
housing/
├── src/
│   ├── main/
│   │   ├── java/com/backend/housing/
│   │   │   ├── application/           # Capa de Aplicación (Use Cases)
│   │   │   │   ├── commands/         # Comandos de negocio
│   │   │   │   ├── mapper/           # Mapeadores DTO ↔ Entidades
│   │   │   │   └── services/          # Casos de uso
│   │   │   ├── domain/               # Capa de Dominio (Core)
│   │   │   │   ├── entity/           # Entidades de negocio
│   │   │   │   ├── events/           # Eventos de dominio
│   │   │   │   ├── exceptions/       # Excepciones de negocio
│   │   │   │   └── valueobjects/      # Objetos de valor
│   │   │   ├── infrastructure/       # Capa de Infraestructura
│   │   │   │   ├── config/           # Configuraciones Spring
│   │   │   │   ├── email/           # Adaptador de envío de emails
│   │   │   │   ├── events/           # Listeners de eventos
│   │   │   │   ├── external/         # Adaptadores externos (Stripe, Supabase)
│   │   │   │   ├── jobs/             # Jobs programados
│   │   │   │   ├── pdf/              # Generadores de PDFs
│   │   │   │   ├── persistence/      # Persistencia (JPA)
│   │   │   │   ├── security/         # Seguridad (JWT)
│   │   │   │   ├── websocket/        # WebSocket
│   │   │   │   └── web/              # Controladores REST
│   │   │   └── HousingApplication.java
│   │   └── resources/
│   └── application.yaml               # Configuración de la aplicación
```

1.2 ¿Por qué Arquitectura Hexagonal?

Ventajas principales:

- Separación de preocupaciones:** El dominio (lógica de negocio) está aislado de la infraestructura (tecnologías externas)
- Testabilidad:** El dominio se puede probar sin depender de bases de datos o APIs externas
- Flexibilidad:** Se pueden cambiar tecnologías (ej: cambiar Stripe por otro proveedor de pagos) sin afectar el dominio
- Mantenibilidad:** La lógica de negocio está centralizada y es fácil de entender

1.3 Patrones de Diseño y Estructuras Utilizadas

1.3.1 Patrón de Arquitectura: Hexagonal (Ports and Adapters)

Este es el patrón principal del proyecto. Divide la aplicación en:

- **Dominio (Core):** Lógica de negocio pura, sin dependencias externas
- **Ports:** Interfaces que definen cómo se comunica el dominio con el exterior
 - **Ports de Entrada (Inbound):** Interfaces que el dominio expone al exterior (ej: `InitiatePaymentUseCase`)
 - **Ports de Salida (Outbound):** Interfaces que el dominio utiliza para comunicarse con sistemas externos (ej: `PaymentProviderPort` , `PaymentRepository`)
- **Adapters:** Implementaciones de los ports que conectan el dominio con tecnologías específicas
 - **Driving Adapters (Izquierda):** Controladores REST, WebSocket, etc., que invocan casos de uso
 - **Driven Adapters (Derecha):** Implementaciones de repositorios (JPA), adaptadores externos (Stripe, Supabase), etc.

1.3.2 Patrón de Diseño: Value Object (Objeto de Valor)

Se utiliza para representar conceptos de negocio que se identifican por su valor, no por un ID. Ejemplos:

- `PropertyId` : Identificador de propiedad
- `Price` : Precio con monto y tipo de transacción
- `Address` : Dirección
- `Coordinates` : Latitud y longitud
- `DateRange` : Rango de fechas
- `PeriodRent` : Monto mensual de arriendo

Características clave:

- Inmutabilidad
- Igualdad por valor (no por identidad)
- Autovalidación en el constructor

1.3.3 Patrón de Diseño: Aggregate (Agregado)

Un Aggregate es un grupo de objetos (entidades y objetos de valor) que se tratan como una unidad para cambios de datos. Ejemplos:

- **Property Aggregate:** Propiedad + sus objetos de valor (`Price`, `Address`, `Coordinates`, etc.)
- **RentalContract Aggregate:** Contrato de arriendo + sus objetos de valor
- **Payment Aggregate:** Pago + sus objetos de valor

Reglas:

- Cada Aggregate tiene una única Entidad Raíz (ej: `Property` es la raíz de su Aggregate)
- Las referencias externas solo pueden ir a la Entidad Raíz
- La consistencia transaccional se garantiza dentro de un Aggregate

1.3.4 Patrón de Diseño: Repository (Repositorio)

Se utiliza para encapsular la lógica de acceso a datos. El dominio define interfaces de repositorio (Ports de Salida), y la infraestructura las implementa usando JPA/Hibernate. Ejemplos:

- `PaymentRepository` : Puerto de salida para persistir pagos
- `RentalContractRepository` : Puerto de salida para persistir contratos

Ventajas:

- El dominio no depende de la tecnología de persistencia
- Facilita el testing (se pueden crear implementaciones mock)

1.3.5 Patrón de Diseño: Service (Capa de Aplicación)

Los servicios de la capa de aplicación implementan los casos de uso (Use Cases). Ejemplos:

- `InitiatePaymentService` : Caso de uso para iniciar un pago
- `HandlePaymentWebhookService` : Caso de uso para manejar webhooks de pagos
- `CreateContractService` : Caso de uso para crear un contrato

Características:

- Coordinan objetos del dominio para realizar operaciones de negocio
- No contienen lógica de negocio (esa va en las entidades)
- Son transaccionales (`@Transactional`)

1.3.6 Patrón de Diseño: Event Driven (Orientado a Eventos)

Se utiliza para desacoplar componentes. El sistema publica eventos de dominio cuando ocurren cosas importantes, y otros componentes reaccionan a estos eventos. Ejemplos:

- `ContractActivatedEvent` : Se publica cuando un contrato se activa
- `PaymentReceivedEvent` : Se publica cuando se recibe un pago
- `NotificationEventListener` : Escucha estos eventos y crea notificaciones

Ventajas:

- Desacoplamiento entre componentes
- Facilita la extensibilidad (se pueden agregar nuevos listeners sin modificar el código existente)

1.3.7 Patrón de Diseño: Factory (Fábrica)

Se utiliza para crear objetos complejos. Las entidades tienen métodos estáticos factory:

- `Property.create(...)` : Crea una propiedad nueva en estado DRAFT
- `Property.reconstitute(...)` : Reconstruye una propiedad desde la persistencia
- `RentalContract.create(...)` : Crea un contrato nuevo
- `Payment.createWithCheckoutSession(...)` : Crea un pago con sesión de checkout

1.3.8 Patrón de Diseño: Adapter (Adaptador)

Se utiliza para integrar sistemas externos, adaptando su API a los ports definidos por el dominio. Ejemplos:

- `StripeCheckoutAdapter` : Implementa `PaymentProviderPort` para conectar con Stripe
- `SupabaseStorageAdapter` : Implementa `ImageStoragePort` para conectar con Supabase Storage
- `EmailServiceAdapter` : Implementa el puerto de envío de emails para conectar con Gmail SMTP

1.3.9 Patrón de Diseño: Strategy (Estrategia)

Aunque no explícitamente, el uso de `PaymentFrequency` (WEEKLY, BIWEEKLY, MONTHLY) y cómo se calculan las fechas de pago es un ejemplo de Strategy.

1.3.10 Patrón de Diseño: State (Estado)

Las entidades tienen estados bien definidos y métodos que cambian de estado según reglas de negocio. Ejemplos:

- `PropertyStatus` : DRAFT → CREATED → PUBLISHED → RENTED/SOLD/DELETED
- `RentalContractStatus` : PAYMENT_PENDING → PAID_NOT_STARTED → ACTIVE → CANCELLATION_PENDING → CANCELLED/TERMINATED/EXPIRED
- `PaymentStatus` : PENDING → SUCCEEDED/FAILED

Cada cambio de estado está encapsulado en métodos de la entidad, que validan que la transición sea válida.

2. Entidades de Dominio Principales

2.1 User (Usuario)

Representa a un usuario del sistema (propietario o arrendatario).

Atributos clave:

- `id` : Identificador único
- `primerNombre` , `segundoNombre` , `primerApellido` , `segundoApellido` : Nombres completos
- `email` : Correo electrónico (único)
- `cedula` : Cédula de identidad
- `phoneNumber` : Número de teléfono
- `profilePictureUrl` : URL de la foto de perfil
- `roles` : Roles del usuario (propietario, arrendatario, etc.)
- `active` : Estado de activación

2.2 Property (Propiedad)

Representa una propiedad inmobiliaria disponible para arriendo o venta.

Atributos clave:

- `id` : `PropertyId` (Objeto de valor)
- `title` : Título de la propiedad
- `description` : Descripción
- `price` : `Price` (Objeto de valor con monto y tipo de transacción: RENT o SALE)
- `typeProperty` : Tipo de propiedad (casa, apartamento, etc.)
- `status` : Estado de la propiedad (`DRAFT` , `CREATED` , `PUBLISHED` , `RENTED` , `SOLD` , `DELETED`)
- `ownerId` : ID del propietario
- `address` : `Address` (Objeto de valor con dirección)
- `coordinates` : `Coordinates` (Objeto de valor con latitud y longitud)
- `rentalTerms` : `RentalTerms` (Objeto de valor con términos de arriendo: frecuencia de pago, depósito, etc.)
- `imageUrls` : Lista de URLs de imágenes
- `numberOfBedrooms` , `numberOfBathrooms` , `areaInSquareMeters` : Características

Métodos de negocio importantes:

- `publish()` : Publica la propiedad (cambia estado a `PUBLISHED`)
- `markAsRented()` : Marca como alquilada
- `markAsSold()` : Marca como vendida
- `disable()` : Deshabilita la propiedad
- `delete()` : Elimina la propiedad (si no está alquilada)

2.3 RentalRequest (Solicitud de Arriendo)

Representa una solicitud de un arrendatario para alquilar una propiedad.

Atributos clave:

- `id` : `RequestId`
- `propertyId` : ID de la propiedad
- `tenantId` : ID del arrendatario

- `ownerId` : ID del propietario
- `period` : `DateRange` (Fecha de inicio y fin del arriendo)
- `proposedRent` : Monto de arriendo propuesto
- `status` : Estado de la solicitud (`PENDING` , `ACCEPTED` , `REJECTED` , `CANCELLED`)
- `createdAt` , `respondedAt` : Fechas de creación y respuesta

Métodos de negocio:

- `accept()` : Acepta la solicitud
- `reject()` : Rechaza la solicitud
- `cancel()` : Cancela la solicitud (solo si está pendiente)

2.4 RentalContract (Contrato de Arriendo)

Representa un contrato de arriendo formal entre propietario y arrendatario.

Atributos clave:

- `id` : `ContractId`
- `propertyId` : ID de la propiedad
- `tenantId` : ID del arrendatario
- `ownerId` : ID del propietario
- `period` : `DateRange` (Periodo del contrato)
- `periodRent` : `PeriodRent` (Monto mensual de arriendo)
- `paymentFrequency` : Frecuencia de pago (`WEEKLY` , `BIWEEKLY` , `MONTHLY`)
- `status` : Estado del contrato:
 - `PAYMENT_PENDING` : Esperando primer pago
 - `PAID_NOT_STARTED` : Pagado pero no ha iniciado el contrato
 - `ACTIVE` : Contrato activo
 - `CANCELLATION_PENDING` : Cancelación programada
 - `CANCELLED` : Cancelado
 - `TERMINATED` : Terminados
 - `EXPIRED` : Expirado
- `actualStartDate` : Fecha real de inicio
- `paymentDueDate` : Fecha límite del próximo pago
- `effectiveCancellationDate` : Fecha efectiva de cancelación

Métodos de negocio importantes:

- `activate(paymentConfirmedDate)` : Activa el contrato después del primer pago
- `startContract()` : Inicia un contrato que estaba pagado pero no iniciado
- `renewPaymentPeriod(newPaymentConfirmedDate)` : Renueva el período de pago después de un pago periódico
- `scheduleCancellation(effectiveDate)` : Programa una cancelación para una fecha futura
- `cancelImmediately()` : Cancela inmediatamente (solo en estados tempranos)
- `expire()` : Marca el contrato como expirado

2.5 Payment (Pago)

Representa un pago realizado por un arrendatario.

Atributos clave:

- `id` : `PaymentId`

- `referenceId` : UUID de la referencia (contrato de arriendo)
- `referenceType` : Tipo de referencia (`RENTAL`)
- `amount` : Monto del pago
- `currency` : Moneda (`COP`)
- `status` : Estado del pago (`PENDING` , `SUCCEEDED` , `FAILED`)
- `method` : Método de pago (`CARD`)
- `period` : Período del pago (formato `yyyy-MM`)
- `checkoutSessionId` : ID de la sesión de Stripe
- `checkoutUrl` : URL del checkout de Stripe
- `createdAt` , `paidAt` : Fechas de creación y pago

Métodos de negocio:

- `markAsSucceeded()` : Marca el pago como exitoso
- `markAsFailed()` : Marca el pago como fallido

3. Entes Externos y su Implementación

3.1 Stripe (Proveedor de Pagos)

Stripe es el proveedor de pagos utilizado para procesar transacciones de arriendo.

Implementación:

- **Adaptador:** `StripeCheckoutAdapter.java` (implementa `PaymentProviderPort`)
- **Configuración:** `StripeConfig.java`
- **Variables de entorno:**
 - `STRIPE_SECRET_KEY` : Clave secreta de Stripe
 - `STRIPE_WEBHOOK_SECRET` : Secreto del webhook de Stripe
 - `SUCCESS_URL` : URL de redirección después de pago exitoso
 - `CANCEL_URL` : URL de redirección después de pago cancelado

Funcionamiento:

1. **Creación de Checkout Session:** Se crea una sesión de pago en Stripe con el monto, moneda, y metadata (`referenceId`, `referenceType`)
2. **Redirección:** El usuario es redirigido a la URL de checkout de Stripe
3. **Webhook:** Stripe envía un evento `checkout.session.completed` al webhook cuando el pago es exitoso

3.2 Supabase Storage (Almacenamiento de Imágenes)

Supabase Storage se utiliza para almacenar las imágenes de las propiedades.

Implementación:

- **Adaptador:** `SupabaseStorageAdapter.java` (implementa `ImageStoragePort`)
- **Configuración:** `SupabaseProperties.java`
- **Variables de entorno:**
 - `SUPABASE_URL` : URL del proyecto Supabase
 - `SUPABASE_KEY` : Clave de API de Supabase
- **Bucket:** `property-images`

Funcionalidades:

- `uploadImage(propertyId, file)` : Sube una imagen al bucket, organizada por `propertyId`

- `deleteImage(imageUrl)` : Elimina una imagen específica
- `deleteAllImages(propertyId)` : Elimina todas las imágenes de una propiedad
- `listImages(propertyId)` : Lista todas las imágenes de una propiedad

3.3 PostgreSQL (Base de Datos)

PostgreSQL es la base de datos relacional utilizada para persistir todas las entidades.

Configuración (application.yaml):

```
spring:
  datasource:
    url: jdbc:postgresql://ep-soft-breeze-aicyu3ti-pooler.c-4.us-east-1.aws.neon.tech/neondb?sslmode=require
    username: neondb_owner
    password: ${DB_PASS}
  jpa:
    hibernate:
      ddl-auto: update # Actualiza el schema automáticamente
    show-sql: true
```

Proveedor de Hosting: Neon Tech (PostgreSQL como servicio)

3.4 Gmail (Servicio de Correo)

Se utiliza Gmail para enviar notificaciones por correo electrónico.

Configuración (application.yaml):

```
spring:
  mail:
    host: smtp.gmail.com
    port: 587
    username: imsharlok@gmail.com
    password: ${MAIL_PASSWORD}
```

Adaptador: `EmailServiceAdapter.java`

4. Flujos de Negocio Principales

4.1 Flujo de Publicación de Propiedad

1. El propietario crea una propiedad en estado `DRAFT`
2. Agrega imágenes, descripción y detalles
3. Publica la propiedad → estado cambia a `PUBLISHED`
4. La propiedad aparece en la lista de propiedades disponibles

4.2 Flujo de Solicitud de Arriendo

1. El arrendatario busca una propiedad publicada
2. Envía una solicitud de arriendo con fechas y monto propuesto
3. El propietario recibe la solicitud
4. El propietario puede:

- **Aceptar:** Se crea un contrato de arriendo
- **Rechazar:** La solicitud se marca como rechazada

5. El arrendatario puede cancelar la solicitud mientras esté pendiente

4.3 Flujo de Creación de Contrato y Pago Inicial

Este es el flujo más complejo y central del sistema.

Pasos:

1. Creación del Contrato:

- El propietario crea un contrato a partir de una solicitud aceptada
- El contrato se crea en estado `PAYMENT_PENDING`
- La propiedad se marca como `RENTED`

2. Iniciación del Pago:

- El arrendatario inicia el pago del primer mes
- Se crea una sesión de checkout en Stripe
- Se guarda un Payment en estado `PENDING` con el `checkoutSessionId`
- El arrendatario es redirigido a Stripe para completar el pago

3. Confirmación del Pago (Webhook):

- Stripe envía un evento `checkout.session.completed` al webhook
- El sistema busca el Payment por `checkoutSessionId`
- Marca el Payment como `SUCCEEDED`
- Activa el contrato:
 - Si la fecha de pago es antes de la fecha de inicio del contrato → estado `PAID_NOT_STARTED`
 - Si la fecha de pago es igual o después → estado `ACTIVE` y se calcula la próxima fecha de pago
- Se publica el evento `ContractActivatedEvent`
- Se envían notificaciones a propietario y arrendatario

4.4 Flujo de Pagos Periódicos

1. El sistema tiene un job programado (`StartPendingContractsScheduler`) que:

- Verifica contratos en estado `PAID_NOT_STARTED`
- Si la fecha de inicio ha llegado, cambia el estado a `ACTIVE`

2. Para contratos activos:

- El arrendatario inicia un pago periódico
- Se crea una nueva sesión de checkout en Stripe
- Cuando el pago es confirmado via webhook:
 - Se marca el Payment como `SUCCEEDED`
 - Se actualiza la `paymentDueDate` del contrato a la próxima fecha según la frecuencia de pago
 - Se publica el evento `PaymentReceivedEvent`
 - Se envían notificaciones

4.5 Flujo de Cancelación de Contrato

Caso 1: Cancelación Temprana (antes de iniciar)

- Si el contrato está en `PAYMENT_PENDING` o `PAID_NOT_STARTED`
- Se cancela inmediatamente → estado `CANCELLED`
- La propiedad vuelve a estar disponible

Caso 2: Cancelación Programada (contrato activo)

- El propietario programa una cancelación para una fecha futura
- El contrato cambia a estado `CANCELLATION_PENDING`
- El arrendatario debe seguir pagando hasta la fecha efectiva
- El job `ProcessPendingCancellationsScheduler` verifica diariamente si la fecha ha llegado
- Cuando llega la fecha, se cancela el contrato → estado `CANCELLED`
- La propiedad vuelve a estar disponible

4.6 Flujo de Vencimiento de Contrato

- El job `ExpireContractsScheduler` se ejecuta diariamente
- Verifica contratos activos cuya fecha de fin ha pasado
- Marca el contrato como `EXPIRED`
- La propiedad vuelve a estar disponible

5. Funcionamiento de los Pagos y Configuraciones

5.1 Flujo Completo de un Pago

1. Iniciación:

- El arrendatario llama al endpoint `/api/payments/initiate` con el `contractId`
- `InitiatePaymentService` valida que el contrato exista y esté en estado correcto
- Crea una sesión de checkout en Stripe via `StripeCheckoutAdapter`
- Crea y guarda un `Payment` en estado `PENDING`
- Retorna la URL de checkout al usuario

2. Procesamiento en Stripe:

- El usuario completa el pago en la página de Stripe
- Stripe redirige al usuario a la `successUrl` o `cancelUrl`

3. Webhook:

- Stripe envía un evento `checkout.session.completed` a `/api/payments/webhook`
- `StripeWebhookController` recibe el evento
- Verifica la firma del webhook usando `STRIPE_WEBHOOK_SECRET`
- Extrae el `checkoutSessionId`
- Llama a `HandlePaymentWebhookService`

4. Confirmación del Pago:

- `HandlePaymentWebhookService` busca el `Payment` por `checkoutSessionId`
- Valida que el `Payment` esté en `PENDING`
- Marca el `Payment` como `SUCCEEDED` y guarda
- Busca el contrato asociado
- Si es el primer pago: activa el contrato

- Si es un pago periódico: renueva el período de pago
- Publica el evento correspondiente

5.2 Configuraciones de Stripe

application.yaml:

```
stripe:
  secret-key: ${STRIPE_SECRET_KEY:sk_test_placeholder}
  webhook-secret: ${STRIPE_WEBHOOK_SECRET:whsec_placeholder}
  success-url: ${SUCCES_URL}
  cancel-url: ${CANCEL_URL}
```

¿Por qué estas configuraciones?

- secret-key : Autentica las solicitudes del servidor a Stripe
- webhook-secret : Verifica que los eventos del webhook realmente vengan de Stripe (seguridad)
- success-url : URL donde se redirige al usuario después de un pago exitoso (frontend)
- cancel-url : URL donde se redirige al usuario si cancela el pago (frontend)

5.3 Metadata en Stripe

Cuando se crea una sesión de checkout, se incluye metadata para identificar el pago:

- referenceId : UUID del contrato
- referenceType : Tipo de referencia (RENTAL)

Esto permite que el webhook sepa a qué contrato pertenece el pago.

6. Jobs Programados (Schedulers)

El proyecto utiliza @EnableScheduling para ejecutar tareas programadas.

6.1 StartPendingContractsScheduler

- **Objetivo:** Iniciar contratos que estaban pagados pero no habían iniciado
- **Frecuencia:** Diariamente
- **Funcionamiento:**
 - Busca contratos en estado PAID_NOT_STARTED
 - Verifica si la fecha de inicio ha llegado
 - Si es así, cambia el estado a ACTIVE y calcula la próxima fecha de pago

6.2 ExpireContractsScheduler

- **Objetivo:** Marcar contratos como expirados cuando su fecha de fin ha pasado
- **Frecuencia:** Diariamente
- **Funcionamiento:**
 - Busca contratos activos o con cancelación pendiente
 - Verifica si la fecha de fin del contrato ha pasado
 - Si es así, marca el contrato como EXPIRED

6.3 ProcessPendingCancellationsScheduler

- **Objetivo:** Procesar cancelaciones programadas

- **Frecuencia:** Diariamente
- **Funcionamiento:**
 - Busca contratos en estado `CANCELLATION_PENDING`
 - Verifica si la fecha efectiva de cancelación ha llegado
 - Si es así, marca el contrato como `CANCELLED`

6.4 SendPaymentRemindersScheduler

- **Objetivo:** Enviar recordatorios de pago
- **Frecuencia:** Configurable
- **Funcionamiento:**
 - Busca contratos activos próximos a su fecha de pago
 - Envía notificaciones por email y/o push

7. Notificaciones y Eventos de Dominio

7.1 Eventos de Dominio

El sistema utiliza eventos de dominio para desacoplar funcionalidades:

- `ContractActivatedEvent` : Se publica cuando un contrato se activa
- `PaymentReceivedEvent` : Se publica cuando se recibe un pago periódico

7.2 Listeners de Eventos

- `NotificationEventListener` : Escucha los eventos y crea notificaciones para los usuarios

7.3 Tipos de Notificaciones

El sistema genera notificaciones para:

- Solicitud de arriendo enviada/aceptada/rechazada
- Contrato activado
- Pago exitoso
- Recordatorio de pago
- Contrato cancelado/terminado/expirado

8. Seguridad

8.1 Autenticación JWT

- **Implementación:** `JwtService.java` y `JwtAuthenticationFilter.java`
- **Flujo:**
 1. El usuario inicia sesión con email y contraseña
 2. El sistema valida las credenciales
 3. Se genera un token JWT y un refresh token
 4. El token JWT se incluye en el encabezado `Authorization: Bearer <token>` en solicitudes subsiguientes
 5. `JwtAuthenticationFilter` valida el token en cada solicitud autenticada

8.2 Autorización

- **Roles:** Los usuarios tienen roles que determinan sus permisos
- **SecurityConfig.java:** Configura qué endpoints son públicos y cuáles requieren autenticación

Endpoints públicos:

- `/api/auth/*` : Registro, login, recuperación de contraseña
- `/api/payments/webhook` : Webhook de Stripe
- `/api/properties/**` (GET): Ver propiedades
- `/swagger-ui/**` : Documentación API
- `/v3/api-docs/**` : OpenAPI specs

Endpoints autenticados:

- Todo lo demás, incluyendo creación de propiedades, contratos, pagos, etc.

8.3 Encriptación de Contraseñas

- Se utiliza `BCryptPasswordEncoder` para encriptar las contraseñas de los usuarios

9. Generación de PDFs

El proyecto genera PDFs para:

1. **Recibo de Pago:** `PaymentReceiptPdfGenerator.java`
2. **Historial de Pagos:** `PaymentHistoryPdfGenerator.java`
3. **Contrato de Arriendo:** `RentalContractPdfGenerator.java`

Librería: `iText 7 ( itext7-core )`

10. Tecnologías Utilizadas

- **Framework:** Spring Boot 4.0.2
- **Java:** 21
- **ORM:** Spring Data JPA (Hibernate)
- **Base de Datos:** PostgreSQL (Neon Tech)
- **Seguridad:** Spring Security + JWT
- **Pagos:** Stripe
- **Almacenamiento:** Supabase Storage
- **Email:** Gmail SMTP
- **PDF:** iText 7
- **Documentación API:** SpringDoc OpenAPI (Swagger)
- **Build Tool:** Maven

11. Configuración del Proyecto (application.yaml)

```
spring:
  application:
    name: housing
  datasource:
    url: jdbc:postgresql://ep-soft-breeze-aicyu3ti-pooler.c-4.us-east-1.aws.neon.tech/neondb?sslmode=require
    username: neondb_owner
    password: ${DB_PASS}
    driver-class-name: org.postgresql.Driver
    hikari:
      maximum-pool-size: 5
  servlet:
    multipart:
```

```

    max-file-size: 50MB
    max-request-size: 100MB
jpa:
  hibernate:
    ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
sql:
  init:
    mode: always
mail:
  host: smtp.gmail.com
  port: 587
  username: imsharlok@gmail.com
  password: ${MAIL_PASSWORD}
  properties:
    mail:
      smtp:
        auth: true
        starttls:
          enable: true
server:
  port: 8080
logging:
  level:
    root: INFO
    org.hibernate.SQL: DEBUG
    org.hibernate.type.descriptor.sql.BasicBinder: TRACE
    org.springframework.security: DEBUG
    org.springframework.web: DEBUG
    com.backend.housing: DEBUG
app:
  default-profile-picture: "https://ui-avatars.com/api/?
name=User&background=6366f1&color=fff&size=200"
supabase:
  url: ${SUPABASE_URL:http://localhost:54321}
  key: ${SUPABASE_KEY:default-key}
  bucket: property-images
stripe:
  secret-key: ${STRIPE_SECRET_KEY:sk_test_placeholder}
  webhook-secret: ${STRIPE_WEBHOOK_SECRET:whsec_placeholder}
  success-url: ${SUCCE_URL}
  cancel-url: ${CANCEL_URL}

```

12. Variables de Entorno Requeridas

- `DB_PASS` : Contraseña de la base de datos PostgreSQL
- `MAIL_PASSWORD` : Contraseña de la cuenta de Gmail para envío de correos

- `SUPABASE_URL` : URL del proyecto Supabase
- `SUPABASE_KEY` : Clave de API de Supabase
- `STRIPE_SECRET_KEY` : Clave secreta de Stripe
- `STRIPE_WEBHOOK_SECRET` : Secreto del webhook de Stripe
- `SUCCESS_URL` : URL de redirección después de pago exitoso
- `CANCEL_URL` : URL de redirección después de pago cancelado

Nota: Esta documentación ignora completamente las funcionalidades de favoritos y chats, según lo solicitado.